

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

SISTEMAS DISTRIBUIDOS

Laboratorio S05

Mexi Banco con Docker Compose

Nombre del alumno:	Yukioayax Canek Gabriel Hernández
Profesor:	Oscar Manuel Ruiz Hurtado
Grupo:	2
Semestre en curso:	2027-1

Ciudad Universitaria, CDMX

Índice

1. Introducción	2
2. Objetivo	2
3. Desarrollo	2
3.1. Levantamiento de Servicios con Compose	2
3.2. Pruebas de Persistencia con Volúmenes	3
3.3. Análisis del Código Fuente (Paso 5)	3
3.4. Estudio de Compose y Dockerfile (Paso 6)	4
4. Resultados	6
5. Análisis de Resultados	6
6. Conclusiones	6

1 Introducción

El presente reporte documenta el desarrollo del Laboratorio S05 enfocado en la orquestación de contenedores utilizando Docker Compose, el objetivo principal es desplegar una aplicación monolítica en Java (Mexi Banco) junto con su base de datos PostgreSQL, analizar el código fuente para comprender su estructura interna y probar la persistencia de datos mediante la manipulación de volúmenes de Docker.

2 Objetivo

Comprender el funcionamiento de `docker-compose.yml` para el levantamiento simultáneo de múltiples servicios, analizar la construcción de imágenes optimizadas a través de un `Dockerfile` de múltiples etapas (*multi-stage builds*) e inspeccionar la arquitectura de un monolito en Java basado en Spring Boot como paso previo a su separación en microservicios.

3 Desarrollo

3.1 Levantamiento de Servicios con Compose

Mediante el comando `docker compose up -build -d`, se procedió a construir la imagen de la aplicación y a levantar los contenedores de la base de datos (`db`) y la aplicación (`app`).

```
#19 resolving provenance for metadata file
#19 DONE 0.1s
[+] up 5/5
✓Image mexi-banco-app Built
✓Network mexibanco Created
✓Volume mexi-banco_mexibanco_datos Created
✓Container mexi-banco-db-1 Healthy
✓Container mexi-banco-app-1 Started
PS D:\VacGab\Desktop\mexi-banco> docker compose ps

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
mexi-banco-app-1	mexi-banco-app	"java -jar app.jar"	app	15 minutes ago	Up 15 minutes (healthy)	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-db-1	postgres:17-alpine	"docker-entrypoint.s..."	db	15 minutes ago	Up 15 minutes (healthy)	5432/tcp

```
PS D:\VacGab\Desktop\mexi-banco> docker compose down
[+] down 3/3
✓Container mexi-banco-app-1 Removed
✓Container mexi-banco-db-1 Removed
✓Network mexibanco Removed
PS D:\VacGab\Desktop\mexi-banco> docker compose up -d
[+] up 3/3
✓Network mexibanco Created
✓Container mexi-banco-db-1 Healthy
✓Container mexi-banco-app-1 Started
PS D:\VacGab\Desktop\mexi-banco> docker compose down -v
[+] down 4/4
✓Container mexi-banco-app-1 Removed
✓Container mexi-banco-db-1 Removed
✓Volume mexi-banco_mexibanco_datos Removed
✓Network mexibanco Removed
PS D:\VacGab\Desktop\mexi-banco> docker compose up -d
#1 [internal] load local bake definitions
#1 reading from stdin 509B 0.1s done
#1 DONE 0.1s
#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 448B 0.0s done
#2 DONE 0.1s
```

Figura 1: Ejecución de Docker Compose y verificación de estado.

Al verificar con `docker compose ps`, ambos servicios muestran el estado **healthy**, indicando que se han iniciado correctamente y que la aplicación ha establecido conexión exitosa con PostgreSQL.

3.2 Pruebas de Persistencia con Volúmenes

Se evaluó el comportamiento de los datos ante el apagado del entorno, al utilizar `docker compose down`, los contenedores se eliminan pero el volumen nombrado `mexibanco_datos` se conserva, manteniendo los saldos intactos al volver a levantar el servicio, por el contrario, al ejecutar `docker compose down -v`, se fuerza la eliminación del volumen de datos.

```
#9 [builder 5/8] COPY .mvn .mvn
#9 CACHED

#10 [builder 8/8] RUN ./mvnw clean package -DskipTests
#10 CACHED

#11 [builder 6/8] RUN ./mvnw dependency:go-offline -B
#11 CACHED

#12 [builder 2/8] WORKDIR /app
#12 CACHED

#13 [builder 3/8] COPY pom.xml .
#13 CACHED

#14 [builder 4/8] COPY mvnw .
#14 CACHED

#15 [builder 7/8] COPY src ./src
#15 CACHED

#16 [stage-1 2/3] WORKDIR /app
#16 CACHED

#17 [stage-1 3/3] COPY --from=builder /app/target/*.jar app.jar
#17 CACHED

#18 exporting to image
#18 exporting layers 0.0s done
#18 exporting manifest sha256:af14b1fda8eae3bc7ded5a5dd19a6b9b5e861d3df0801c3bc716d4b4209f92d7 0.0s done
#18 exporting config sha256:ff78005bd203911abbfdad3b46d98533c3c8301359ed1ca396037b4ce591f859 0.0s done
#18 exporting attestation manifest sha256:e8e59f47da71cfa016cc0a4fd63e689b5dd646c50727fd7869a2ef1ae7e50640
#18 exporting attestation manifest sha256:e8e59f47da71cfa016cc0a4fd63e689b5dd646c50727fd7869a2ef1ae7e50640 0.1s done
#18 exporting manifest list sha256:f4b6e07eb37944e36bd1fa9e6fc15009e7d572c7b81741cfc54539257b0e0cd9 0.1s done
#18 naming to docker.io/library/mexi-banco-app:latest
#18 naming to docker.io/library/mexi-banco-app:latest 0.0s done
#18 unpacking to docker.io/library/mexi-banco-app:latest 0.0s done
#18 DONE 0.4s

#19 resolving provenance for metadata file
#19 DONE 0.0s
[+] up 5/5
✔Image mexi-banco-app Built
✔Network mexibanco Created
✔Volume mexi-banco_mexibanco_datos Created
✔Container mexi-banco-db-1 Healthy
✔Container mexi-banco-app-1 Started
```

Figura 2: Apagado de contenedores y eliminación explícita de volúmenes con la bandera `-v`.

Tras este comando, levantar el entorno genera un volumen nuevo y vacío, perdiendo permanentemente cualquier operación previa registrada en Postman.

3.3 Análisis del Código Fuente (Paso 5)

Al inspeccionar el proyecto Java, se identificó una arquitectura fuertemente modularizada, cada módulo (cuenta, movimiento, spei, transferencia) implementa un patrón estricto de capas, a continuación se muestran fragmentos clave que ilustran esta estructura:

1. Capa de Controlador (Endpoints HTTP): Expone las rutas REST e interactúa directamente con los servicios, sin contener reglas de negocio.

CuentaController.java

```
@RestController @RequestMapping("/cuentas") public class CuentaController
private final CuentaService cuentas;
@PostMapping @ResponseStatus(HttpStatus.CREATED) public Cuenta abrir(@Valid
@RequestBody AltaCuenta datos) return cuentas.abrir(datos.clabe(),
datos.titular(), datos.saldoInicial());
```

2. Capa de Servicio (Lógica y Transacciones): Centraliza las reglas del banco, en operaciones complejas, la anotación `@Transactional` garantiza la atomicidad (todo o nada), lo cual es posible porque actualmente todos los módulos comparten la misma base de datos.

TransferenciaService.java

```
@Service public class TransferenciaService // ... @Transactional public
Transferencia transferir(String claveOrigen, String claveDestino,
BigDecimal monto) // Cargo y abono atómico dentro del monolito
Cuenta origen = cuentas.cargar(claveOrigen, monto); Cuenta destino =
cuentas.abonar(claveDestino, monto);
Movimiento cargo = movimientos.registrar(origen.getClabe(), ...); Movimiento
abono = movimientos.registrar(destino.getClabe(), ...);
return transferencias.save(new Transferencia(claveOrigen, claveDestino,
monto));
```

3.4 Estudio de Compose y Dockerfile (Paso 6)

El archivo `docker-compose.yml` se encarga de la orquestación, su configuración más crítica es el `healthcheck` acoplado al `depends_on`.

```
docker-compose.yml X
docker-compose.yml
1 # Con un solo comando: docker compose up --build
2 # Lo que van a ver correr (S05, "de compose a orquestacion"):
3 # - 'app' resuelve 'db' por nombre (DNS embebido de Compose), sin ninguna IP en ningun lado.
4 # - 'app' espera a que 'db' este listo de verdad (depends_on + healthcheck), no solo que exista.
5 # - Las variables de entorno son la configuracion (factor III, ya visto en S04).
6 # - 'db' persiste en un volumen nombrado: si tiran el contenedor y lo vuelven a subir, los
7 #   datos siguen ahí; si tiran el volumen, no.
8 services:
9   db:
10     image: postgres:17-alpine
11     environment:
12       POSTGRES_DB: mexibanco
13       POSTGRES_USER: mexibanco
14       POSTGRES_PASSWORD: mexibanco
15     volumes:
16       - mexibanco_datos:/var/lib/postgresql/data
17     healthcheck:
18       test: ["CMD-SHELL", "pg_isready -U mexibanco -d mexibanco"]
19       interval: 5s
20       timeout: 3s
21       retries: 5
22
23   app:
24     build: .
25     ports:
26       - "8080:8080"
27     environment:
28       DB_HOST: db
29       DB_PORT: "5432"
30       DB_NAME: mexibanco
31       DB_USER: mexibanco
32       DB_PASSWORD: mexibanco
33     depends_on:
34       db:
35         condition: service_healthy
36     healthcheck:
37       test: ["CMD-SHELL", "wget -qO- http://localhost:8080/actuator/health | grep -q UP || exit 1"]
38       interval: 10s
39       timeout: 3s
40       retries: 5
41
42 networks:
43   default:
44     name: mexibanco
45
46 volumes:
47   mexibanco_datos:
48
```

Figura 3: Archivo docker-compose.yml mostrando healthchecks y dependencias.

El servicio `app` no arranca simplemente porque `db` exista, sino que espera a que la condición `service_healthy` se cumpla, asegurando que PostgreSQL esté listo para recibir peticiones. Por otro lado, la construcción de la imagen se define en el `Dockerfile` utilizando dos etapas (*multi-stage build*).

```
Dockerfile X
Dockerfile
1 # ---- ETAPA 1: CONSTRUCCION ----
2 FROM eclipse-temurin:21-jdk-alpine AS builder
3 WORKDIR /app
4 COPY pom.xml .
5 COPY mvnw .
6 COPY .mvn .mvn
7 RUN ./mvnw dependency:go-offline -B
8 COPY src ./src
9 RUN ./mvnw clean package -DskipTests
10
11 # ---- ETAPA 2: EJECUCION ----
12 FROM eclipse-temurin:21-jre-alpine
13 WORKDIR /app
14 COPY --from=builder /app/target/*.jar app.jar
15 EXPOSE 8080
16
17 ENTRYPOINT ["java", "-jar", "app.jar"]
18
```

Figura 4: Archivo Dockerfile en dos etapas.

La primera etapa (**builder**) utiliza el JDK completo para descargar dependencias y compilar el `.jar`. La segunda etapa usa únicamente el JRE, copiando el binario compilado.

4 Resultados

Se logró desplegar exitosamente la aplicación Mexi Banco y su base de datos de manera coordinada. Las pruebas en Postman confirmaron que los contenedores procesan correctamente las peticiones. Asimismo, se comprobó experimentalmente la diferencia crítica entre bajar los contenedores preservando el volumen y destruyéndolo (`-v`), observando la pérdida total de datos en el segundo caso.

5 Análisis de Resultados

La práctica evidenció la importancia de condicionar el inicio de los contenedores; sin la directiva `service_healthy` en el archivo Compose, la aplicación de Spring Boot habría fallado al intentar conectarse a un PostgreSQL que aún estaba en proceso de inicialización, en cuanto al `Dockerfile`, el uso de múltiples etapas demostró ser una técnica esencial: aislar el entorno de compilación (JDK) del de ejecución (JRE) no solo reduce significativamente el tamaño final de la imagen, sino que disminuye su superficie de ataque al no incluir código fuente ni herramientas de desarrollo en producción.

Finalmente, el análisis del código Java reveló los retos futuros: operaciones como **transferir**, que actualmente se aseguran con una simple etiqueta `@Transactional` en el monolito, requerirán soluciones de consistencia eventual cuando las cuentas y los movimientos se separen en contenedores distintos.

6 Conclusiones

Docker Compose simplifica drásticamente el despliegue de entornos multicontenedor, permitiendo definir redes, volúmenes y dependencias lógicas en un solo archivo declarativo, comprendí que el ciclo de vida de los datos está directamente ligado a la gestión de los volúmenes, siendo imperativo manejar comandos como `down -v` con extrema precaución en entornos productivos, por último, la inspección del código monolítico sirvió para sentar las bases teóricas de por qué migrar a microservicios exige un rediseño profundo en la comunicación y las transacciones de las aplicaciones distribuidas.